

BE CSE-AIML | Semester V | Osmania University

DATA MINING

Mid-1 and Mid-2 — Complete Answer Guide

All questions answered with detailed explanations, examples, tables, formulas and algorithms

Han, Kamber & Pei — Data Mining: Concepts and Techniques, 3rd Edition

Topic	Content
Mid-1 Part A	Data Mining, Data Visualization, Hidden Pattern, Frequent Pattern Mining, Support (5x2=10M)
Mid-1 Q2	Major Issues in Data Mining — 5 categories with detailed explanation (10M)
Mid-1 Q3	Apriori Algorithm — complete L1,L2,L3 trace on given 9-transaction dataset (10M)
Mid-1 Q4	FP-Growth — FP-tree construction step-by-step + conditional pattern mining (10M)
Mid-2 Part A	Supervised/Unsupervised Learning, Dataset, Bayes Theorem, Neural Network Layers (5x2=10M)
Mid-2 Q1	K-Means on $D=\{2,4,10,12,3,20,30,11,25\}$ — 3-iteration worked solution (10M)
Mid-2 Q2	DBSCAN — concept, algorithm steps, point types, example, advantages (10M)
Mid-2 Q3	Decision Tree — ID3, entropy, information gain, Play Tennis example (10M)

Class Test-I | Subject: DATA MINING

Class: BE CSE-AIML WP | Semester V | Max Marks: 30 | Date: 24-03-2026

PART A — Answer All Questions (5 x 2 = 10 Marks)

2 Marks

Q1a. What is Data Mining?

Data Mining is the process of discovering interesting, non-trivial, implicit, previously unknown, and potentially useful patterns or knowledge from large amounts of data stored in databases, data warehouses, or other information repositories. It is also known as Knowledge Discovery in Databases (KDD).

Key tasks in data mining:

- **Classification** — assigning data objects to predefined classes (e.g., spam/not spam)
- **Clustering** — grouping similar objects without predefined labels (unsupervised)
- **Association Rule Mining** — finding items that frequently co-occur (e.g., diapers → beer)
- **Regression** — predicting a numeric value from attributes
- **Outlier Detection** — finding data objects that deviate significantly from the norm

KDD Process: Raw Data → Data Cleaning → Integration → Selection → Transformation → Data Mining → Pattern Evaluation → Knowledge Presentation
Example: A bank mines credit card transaction records to find fraud patterns. Example: Amazon mines purchase history to recommend products ("customers also bought").

2 Marks

Q1b. What is Data Visualization?

Data Visualization is the graphical or visual presentation of data and data mining results to allow users to understand patterns, trends, clusters, and outliers intuitively. It leverages the human visual system — controlled by the eyes and brain — as a powerful parallel processing engine.

Form of Visualization	What it shows	Example tool
Scatter plots	Relationship between two numeric attributes	SAS Enterprise Miner
Boxplots	Distribution, median, quartiles, outliers of a variable	StatSoft
3D data cubes	Multi-dimensional data analysis at once	OLAP systems
Association rule graph	Which items co-occur — nodes and weighted edges	MineSet
Decision tree diagram	Classification rules in tree form	MineSet, Clementine
Cluster grouping	Visual separation of cluster memberships	IBM Intelligent Miner

Types of visual data mining: (1) **Data visualization** — display raw data as charts. (2) **Mining result visualization** — display discovered patterns. (3) **Mining process visualization** — show how data flows through preprocessing and mining. (4) **Interactive visual mining** — user guides the mining process using visual tools.

2 Marks

Q1c. What is Hidden Pattern?

Hidden Patterns are regularities, relationships, trends, or structures embedded in large datasets that are NOT immediately visible or obvious from a direct examination of the raw data. They require data mining techniques to uncover.

Characteristics of hidden patterns:

- **Non-trivial** — not obvious or expected; would not be found by simple inspection
- **Implicit** — buried in large volumes of data, invisible to human analysts
- **Previously unknown** — genuinely new knowledge, not already documented
- **Potentially useful** — actionable for decision-making or prediction

Example 1: Beer-Diaper Association Supermarket discovers: customers who buy diapers on Friday evenings also tend to buy beer. This hidden pattern was buried in millions of transaction records. Action: Place beer near diapers on Fridays → increased sales. Example 2: Network Intrusion A sequence of seemingly harmless network events across thousands of log entries, when viewed together, reveals a coordinated intrusion pattern invisible to human analysts.

2 Marks

Q1d. What is Frequent Pattern Mining?

Frequent Pattern Mining discovers patterns — itemsets, subsequences, or substructures — that appear frequently in a given dataset. A pattern is "frequent" if its occurrence count meets or exceeds a user-specified minimum support threshold (min_sup).

Type	Definition	Example
Frequent Itemset	Set of items co-occurring frequently in transactions	{milk, bread, butter} appears in 40% of grocery transactions
Frequent Subsequence	Ordered sequence of events occurring frequently	"Buy PC" → "Buy printer" → "Buy cartridge" — common customer journey
Frequent Substructure	Frequently occurring subgraphs/subtrees	A specific chemical substructure in many active drug compounds

Frequent Itemset: An itemset X is frequent if $\text{Support}(X) \geq \text{min_sup}$
Apriori Property: All non-empty subsets of a frequent itemset are also frequent.
Applications: Market basket analysis, web click-stream analysis, DNA sequence analysis, fraud detection

2 Marks

Q1e. What is Support?

Support of an itemset or association rule is the proportion (or count) of transactions in the database that contain the itemset. It measures how frequently the pattern appears across the entire dataset.

Support count of itemset X : $\text{count}(X)$ = number of transactions containing X
Support (relative): $\text{Support}(X) = \text{count}(X) / \text{Total number of transactions}$
For association rule $X \Rightarrow Y$: $\text{Support}(X \Rightarrow Y) = \text{Support}(X \cup Y) = P(X \cup Y)$
Confidence: $\text{Confidence}(X \Rightarrow Y) = \text{Support}(X \cup Y) / \text{Support}(X)$
Example: Database has 10 transactions. {milk, bread} appears in 4 transactions. Support count = 4
Support = $4/10 = 40\%$
If {milk} appears in 6 transactions: $\text{Confidence}(\text{milk} \Rightarrow \text{bread}) = 40\% / 60\% = 66.7\%$

Minimum Support (min_sup): User-specified threshold. Patterns with Support $\geq$ min_sup are called frequent. Setting too low $\rightarrow$ too many patterns; too high $\rightarrow$ miss rare but important patterns.

PART B — Answer Any Two Questions (2 x 10 = 20 Marks)

10 Marks

Q2. Explain Major Issues in Data Mining

Data mining faces numerous challenges spanning its full lifecycle. The major issues are grouped into five categories:

1. Mining Methodology and User Interaction

Sub-issue	Description	Example / Solution
Mining diverse data types	Need algorithms for relational, text, image, video, spatial, graph, stream data	K-means for numeric, CLUTO for text, gSpan for graphs
Diversity of knowledge	Different tasks need different outputs: rules, trees, clusters, sequences	Association rules for retail; clustering for biology; classification for fraud
Interactive mining	Users should guide the mining process, provide feedback, and refine results	Constraint-based mining lets users restrict search space
Background knowledge	Use concept hierarchies, ontologies, and domain knowledge	Use product taxonomy (dairy -> milk) to find rules at multiple abstraction levels
Handling noise and uncertainty	Real data is incomplete, noisy, and inconsistent	Robust algorithms, outlier removal, data cleaning in preprocessing

2. Performance Issues

- **Efficiency and scalability:** Algorithms must handle millions of records and thousands of attributes. Apriori needs optimisation for large databases (FP-Growth, sampling, partitioning).
- **Parallel and distributed computing:** Mining must scale to clusters, multi-core CPUs, and cloud environments. MapReduce and Spark-based mining frameworks address this.
- **Incremental updating:** When new data arrives, discovered patterns should be updated without re-mining from scratch (e.g., new transactions added to transaction database).
- **Handling high-dimensional data:** With thousands of attributes, algorithms fail due to the "curse of dimensionality." Dimensionality reduction (PCA, feature selection) is needed first.

3. Issues with Diverse and Complex Data

Data Type	Challenge	Mining Approach
Heterogeneous databases	Different schemas, formats, and semantics across sources	Data integration, ETL pipelines, federated mining
Stream data	Continuous, infinite, one-pass — cannot store all data	Sliding windows, tilted-time windows, single-scan algorithms
Biological sequences	Very long, high-dimensional, complex hidden semantics	Alignment (BLAST), Hidden Markov Models, motif discovery
Web and social data	Unstructured text, hyperlinks, dynamic content	Web content mining, web structure mining, sentiment analysis
Spatiotemporal data	Both location and time dependencies	Trajectory mining, spatial clustering, movement patterns

4. Issues with Mining Results

- **Interestingness of patterns:** Not all discovered patterns are useful. Need objective measures (support, confidence, lift) and subjective measures (unexpectedness, novelty).
- **Pattern presentation:** Results must be presented in an understandable form — visual charts, natural language rules, decision tree diagrams.
- **Measuring accuracy:** Need metrics: accuracy / F1-score for classification; silhouette coefficient for clustering; lift / leverage for association rules.
- **Conflicting patterns:** Two discovered rules may contradict each other. Need conflict resolution and rule pruning mechanisms.

5. Privacy, Security, and Social Issues

- **Privacy:** Mining personal data (health records, financial data, location) without consent violates privacy. Techniques: k-anonymity, l-diversity, differential privacy, data perturbation.
- **Security:** Mining tools can be misused for surveillance, identity theft, or discriminatory profiling.
- **Invisible mining:** Software (search engines, email filters, recommender systems) mines user data without explicit awareness — raises ethical questions.
- **Ownership:** Discovered knowledge raises intellectual property questions — who owns patterns mined from shared data?

Summary: 5 categories — Methodology & Interaction | Performance | Complex Data | Result Quality | Privacy & Ethics

10 Marks

Q3. Explain Apriori Algorithm and apply on the given dataset

Apriori Algorithm (Agrawal & Srikant, 1994) is the foundational algorithm for mining frequent itemsets for Boolean association rules. It uses the **Apriori Property (anti-monotone)**: all non-empty subsets of a frequent itemset must also be frequent. Equivalently, if any subset of an itemset is infrequent, that itemset itself cannot be frequent.

Apriori Property: if $\{A, B\}$ is NOT frequent, then $\{A, B, C\}$ is also NOT frequent
 Algorithm outline: Scan 1: Find frequent 1-itemsets L_1 For $k = 2, 3, \dots$ while $L(k-1)$ is not empty: $C_k = \text{apriori_gen}(L(k-1))$ -- join + prune step Scan database to count support of each candidate in C_k $L_k = \{c \in C_k \mid \text{support}(c) \geq \text{min_sup}\}$ Return all L_k Join step: $L(k-1)$ joined with $L(k-1)$ -- two itemsets joined if they share first $k-2$ items Prune step: Remove candidate if any $(k-1)$ -subset is not in $L(k-1)$

Given Transaction Database with $\text{min_sup} = 2$ (transactions):

TID	Items Purchased
T100	I1, I2, I5
T101	I2, I4
T102	I2, I3
T103	I1, I2, I4
T104	I1, I3
T105	I2, I3
T106	I1, I3

T107	I1, I2, I3, I5
T108	I1, I2, I3

SCAN 1: Generate C1 and compute support -> L1 (frequent 1-itemsets)

C1 Itemset	Transactions containing it	Support Count	min_sup= 2?	In L1?
{I1}	T100, T103, T104, T106, T107, T108	6	YES	YES
{I2}	T100, T101, T102, T103, T105, T107, T108	7	YES	YES
{I3}	T102, T104, T105, T106, T107, T108	6	YES	YES
{I4}	T101, T103	2	YES	YES
{I5}	T100, T107	2	YES	YES

L1 = { {I1}:6, {I2}:7, {I3}:6, {I4}:2, {I5}:2 } (ALL 5 items are frequent)

SCAN 2: Generate C2 from L1 (join L1 x L1) -> count support -> L2

C2 Candidate	Support Count	Transactions	In L2?
{I1, I2}	4	T100, T103, T107, T108	YES
{I1, I3}	4	T104, T106, T107, T108	YES
{I1, I4}	1	T103 only	NO — PRUNED (1 < 2)
{I1, I5}	2	T100, T107	YES
{I2, I3}	4	T102, T105, T107, T108	YES
{I2, I4}	2	T101, T103	YES
{I2, I5}	2	T100, T107	YES
{I3, I4}	0	None	NO — PRUNED (0 < 2)
{I3, I5}	1	T107 only	NO — PRUNED (1 < 2)
{I4, I5}	0	None	NO — PRUNED (0 < 2)

L2 = { {I1,I2}:4, {I1,I3}:4, {I1,I5}:2, {I2,I3}:4, {I2,I4}:2, {I2,I5}:2 }

SCAN 3: Generate C3 from L2 (join + Apriori prune) -> count -> L3

Join step — pairs from L2 sharing same (k-2) = 1 prefix item:

- {I1,I2} ∪ {I1,I3} → {I1,I2,I3} — verify subset {I2,I3} ∈ L2? YES ✓ → KEEP
- {I1,I2} ∪ {I1,I5} → {I1,I2,I5} — verify subset {I2,I5} ∈ L2? YES ✓ → KEEP
- {I1,I3} ∪ {I1,I5} → {I1,I3,I5} — verify subset {I3,I5} ∈ L2? NO ✗ → PRUNED
- {I2,I3} ∪ {I2,I4} → {I2,I3,I4} — verify subset {I3,I4} ∈ L2? NO ✗ → PRUNED
- {I2,I3} ∪ {I2,I5} → {I2,I3,I5} — verify subset {I3,I5} ∈ L2? NO ✗ → PRUNED
- {I2,I4} ∪ {I2,I5} → {I2,I4,I5} — verify subset {I4,I5} ∈ L2? NO ✗ → PRUNED

C3 after pruning = { {I1,I2,I3}, {I1,I2,I5} }

C3 Candidate	Support Count	Transactions	In L3?
{I1, I2, I3}	2	T107, T108	YES
{I1, I2, I5}	2	T100, T107	YES

$L3 = \{ \{I1, I2, I3\}:2, \{I1, I2, I5\}:2 \}$

SCAN 4: Generate C4 -> empty -> STOP

Join $\{I1, I2, I3\} \cup \{I1, I2, I5\} \rightarrow \{I1, I2, I3, I5\}$. Prune check: subset $\{I1, I3, I5\} \in L3$? NO $\rightarrow$ PRUNED. C4 = empty. Algorithm terminates.

APRIORI FINAL RESULT (min_sup = 2): L1: {I1}:6 {I2}:7 {I3}:6 {I4}:2 {I5}:2 L2: {I1, I2}:4 {I1, I3}:4 {I1, I5}:2 {I2, I3}:4 {I2, I4}:2 {I2, I5}:2 L3: {I1, I2, I3}:2 {I1, I2, I5}:2

10 Marks

Q4. Explain FP-Growth Algorithm using the same dataset

FP-Growth (Frequent Pattern Growth) (Han et al., 2000) mines frequent itemsets WITHOUT candidate generation. It compresses the transaction database into a compact **FP-tree**, then mines patterns directly using a divide-and-conquer strategy. It is typically 10-100x faster than Apriori.

FP-Growth vs Apriori: Apriori: multiple database scans + large candidate sets
 FP-Growth: only 2 database scans + no candidate generation
 FP-Growth steps: Step 1: Scan DB once -> find frequent 1-items, sort by frequency descending
 Step 2: Scan DB again -> insert each transaction (reordered) into FP-tree
 Step 3: Mine FP-tree bottom-up using conditional pattern bases

STEP 1 — First scan: Build L1, sort by frequency descending

From the same database: $I2(7) > I1(6) = I3(6) > I4(2) = I5(2)$

Frequency order for tree insertion: I2, I1, I3, I4, I5

STEP 2 — Reorder each transaction and insert into FP-tree

TID	Original	Reordered (by freq desc)
T100	I1, I2, I5	I2, I1, I5
T101	I2, I4	I2, I4
T102	I2, I3	I2, I3
T103	I1, I2, I4	I2, I1, I4
T104	I1, I3	I1, I3
T105	I2, I3	I2, I3
T106	I1, I3	I1, I3
T107	I1, I2, I3, I5	I2, I1, I3, I5
T108	I1, I2, I3	I2, I1, I3

STEP 3 — Resulting FP-tree structure

Header Table: FP-tree: Item | Count Root I2 | 7 -----> I2 : 7 I1 | 6 |-- I1 : 4 I3 | 6 | |-- I5 : 1 (T100) I4 | 2 | |-- I4 : 1 (T103) I5 | 2 | |-- I3 : 2 (T107, T108) | |-- I5 : 1 (T107) (Each item in header |-- I4 : 1 (T101) has node-links |-- I3 : 2 (T102, T105) connecting all its I1 : 2 (T104, T106) nodes in the tree) |-- I3 : 2 (T104, T106) Shared prefix saves space: T107,T108 share I2->I1->I3 path (count=2)

STEP 4 — Mine FP-tree: conditional pattern bases (bottom-up)

Item	Conditional Pattern Base	Conditional FP-tree	Frequent Patterns Generated
I5	{I2,I1}:1 AND {I2,I1,I3}:1	I2:2, I1:2 (both count=2)	{I5}:2, {I2,I5}:2, {I1,I5}:2, {I1,I2,I5}:2
I4	{I2,I1}:1 AND {I2}:1	I2:2 (only I2 meets min_sup)	{I4}:2, {I2,I4}:2
I3	{I2,I1}:2, {I2}:2, {I1}:2	I2:4, I1:4 (and I2,I1 together:2)	{I3}:6, {I2,I3}:4, {I1,I3}:4, {I1,I2,I3}:2
I1	{I2}:4 (from I2->I1 branch)	I2:4	{I1}:6, {I2,I1}:4
I2	No prefix (child of root)	Single-path: empty	{I2}:7

FP-Growth FINAL RESULT (same as Apriori — same patterns, found without candidate generation): Single: {I1}:6 {I2}:7 {I3}:6 {I4}:2 {I5}:2 Pairs: {I1,I2}:4 {I1,I3}:4 {I1,I5}:2 {I2,I3}:4 {I2,I4}:2 {I2,I5}:2 Triples: {I1,I2,I3}:2 {I1,I2,I5}:2

Why FP-Growth is faster: (1) Only 2 DB scans vs k+1 in Apriori. (2) No candidate generation — saves time and memory. (3) FP-tree compresses the database — shared prefixes reduce memory. (4) Divide-and-conquer avoids redundant subset checking.

Class Test-II | Subject: DATA MINING

Class: BE CSE-AIML WP | Semester V | Max Marks: 30 | Date: 05-05-2026

PART A — Answer All Questions (5 x 2 = 10 Marks)

2 Marks

Q1. What is supervised learning and list 3 algorithms?

Supervised learning is a machine learning paradigm where a model is trained on a **labelled dataset** — each training example has an input (feature vector) and its corresponding correct output (class label or numeric value). The model learns a mapping from inputs to outputs, then predicts outputs for new unseen data.

Algorithm	Type	How it works	Example use
Decision Tree (ID3/C4.5)	Classification	Recursively splits data on attributes with highest information gain	Email spam detection
Naive Bayes Classifier	Classification	Applies Bayes theorem with feature independence assumption	Text/document categorization
Support Vector Machine (SVM)	Classification/Regression	Finds optimal hyperplane maximally separating classes	Image recognition, bioinformatics

2 Marks

Q2. What is unsupervised learning and list 3 algorithms?

Unsupervised learning is a machine learning paradigm where a model is trained on an **unlabelled dataset** — no predefined class labels are provided. The algorithm discovers inherent structure, patterns, or groupings within the data by itself.

Algorithm	Task	How it works	Example
K-Means Clustering	Clustering	Partitions data into k clusters by minimizing within-cluster distance	Customer segmentation
DBSCAN	Clustering	Groups points by density; marks outliers as noise — no k needed	Geographic cluster detection
Hierarchical Clustering (AGNES)	Clustering	Agglomerative — merges closest clusters bottom-up, builds dendrogram	Gene expression analysis

2 Marks

Q3. What is a dataset? Explain training data and test data.

Dataset: A structured collection of data objects (instances / records), where each object is described by a set of **attributes (features)** and usually a **class label**. It forms the foundation for building and evaluating machine learning models.

Aspect	Training Data	Test Data
Definition	Subset used to build/fit the model	Subset used to evaluate model performance
Labels	Known — model learns from these	Known but hidden during prediction
Model exposure	Model sees this during learning	Model never sees this during training
Typical split	70-80% of total data	20-30% of total data

Goal	Minimise training (in-sample) error	Estimate generalisation (out-of-sample) error
Risk	Overfitting — model memorises training data	Underfitting — if too little test data used

Cross-validation: To get a reliable estimate, k-fold cross-validation divides the data into k equal parts; trains on k-1 parts, tests on 1 part, repeats k times, and averages results.

2 Marks

Q4. Explain Bayes Theorem.

Bayes' Theorem describes the probability of a hypothesis H given observed evidence E. It relates conditional and prior probabilities, allowing us to update our belief about H based on new evidence E.

Bayes' Theorem: $P(H | E) = [P(E | H) \times P(H)] / P(E)$ $P(H | E)$ = Posterior probability of H given evidence E (what we want) $P(E | H)$ = Likelihood: probability of seeing E if H is true $P(H)$ = Prior probability of H (before seeing evidence) $P(E)$ = Marginal probability of E (normalising constant) Naive Bayes Classification (uses feature independence assumption): $P(C_i | X) = P(X | C_i) \times P(C_i) / P(X)$ $P(X | C_i)$ = product of $P(x_k | C_i)$ for each feature x_k (independence) Predict: class = $\text{argmax}_i [P(C_i) \times \text{product}_k P(x_k | C_i)]$

Example: Spam detection $P(\text{spam}) = 0.4$, $P(\text{ham}) = 0.6$ $P(\text{'win'} | \text{spam}) = 0.8$, $P(\text{'win'} | \text{ham}) = 0.1$
 $P(\text{'win'}) = P(\text{'win'}|\text{spam}) \times P(\text{spam}) + P(\text{'win'}|\text{ham}) \times P(\text{ham}) = 0.8 \times 0.4 + 0.1 \times 0.6 = 0.32 + 0.06 = 0.38$
 $P(\text{spam} | \text{'win'}) = (0.8 \times 0.4) / 0.38 = 0.32 / 0.38 = 0.842 \Rightarrow 84.2\%$ probability this email is spam given it contains 'win'

2 Marks

Q5. Explain input layer, hidden layer, output layer, activation function and bias.

A **Multilayer Feed-Forward Neural Network** (also called MLP — Multi-Layer Perceptron) consists of three types of layers:

Component	Role	Details	Example
Input Layer	Receives raw feature data	One neuron per feature; no computation; passes values forward	28x28 image -> 784 input neurons
Hidden Layer(s)	Learn intermediate feature representations	One or more layers; most computation; applies weights + activation	2 hidden layers with 128 neurons each
Output Layer	Produces final prediction	Activation depends on task: sigmoid (binary), softmax (multiclass), linear (regression)	10 neurons for digit 0-9 classification
Activation Function	Introduces non-linearity	Without it, network = linear model regardless of depth	ReLU: $\max(0, z)$, Sigmoid: $1/(1+e^{-z})$, tanh
Bias	Shifts activation threshold	Constant added to weighted sum; prevents all outputs = 0 when all inputs = 0	b in: $z = w_1x_1 + \dots + w_nx_n + b$

Computation at each neuron: Weighted sum: $z = w_1 \cdot x_1 + w_2 \cdot x_2 + \dots + w_n \cdot x_n + b$
Activation output: $a = f(z)$ Common activation functions: ReLU: $f(z) = \max(0, z)$ -- most used in hidden layers Sigmoid: $f(z) = 1 / (1 + e^{(-z)})$ -- output layer for binary classification Softmax: $f(z_i) = e^{z_i} / \sum(e^{z_j})$ -- output layer for multiclass tanh: $f(z) = (e^z - e^{(-z)}) / (e^z + e^{(-z)})$ -- range (-1, 1)

PART B — Answer Any Two Questions (2 x 10 = 20 Marks)

10 Marks

Q1. Explain K-Means Algorithm and apply on $D = \{2, 4, 10, 12, 3, 20, 30, 11, 25\}$

K-Means Clustering partitions n objects into exactly k non-overlapping clusters. It minimizes the **within-cluster sum of squared distances (WCSS)** between each point and its cluster centroid. It is the most widely used clustering algorithm.

Objective function (minimize): $WCSS = \text{Sum over } k \text{ clusters } [\text{Sum over } x_i \text{ in } C_k: |x_i - \mu_k|^2]$
 Algorithm steps: 1. Choose k ; randomly initialize k centroids μ_k 2. ASSIGN: assign each point x_i to nearest centroid: $c(i) = \text{argmin}_k |x_i - \mu_k|$ 3. UPDATE: recompute centroids: $\mu_k = \text{mean of all } x_i \text{ in cluster } k$ 4. REPEAT steps 2-3 until centroids do not change (convergence)

Applying K-Means with $k=2$ on $D = \{2, 4, 10, 12, 3, 20, 30, 11, 25\}$: (9 data points, 1-dimensional)

Initial centroids: $C1 = 2, C2 = 20$ (first value and a middle-range value)

ITERATION 1 — Assign each point to nearest centroid ($C1=2, C2=20$)

Point x	$ x - C1=2 $	$ x - C2=20 $	Nearest centroid	Cluster
2	$ 2-2 = 0$	$ 2-20 = 18$	$C1 (0 < 18)$	C1
4	$ 4-2 = 2$	$ 4-20 = 16$	$C1 (2 < 16)$	C1
10	$ 10-2 = 8$	$ 10-20 = 10$	$C1 (8 < 10)$	C1
12	$ 12-2 = 10$	$ 12-20 = 8$	$C2 (10 > 8)$	C2
3	$ 3-2 = 1$	$ 3-20 = 17$	$C1 (1 < 17)$	C1
20	$ 20-2 = 18$	$ 20-20 = 0$	$C2 (18 > 0)$	C2
30	$ 30-2 = 28$	$ 30-20 = 10$	$C2 (28 > 10)$	C2
11	$ 11-2 = 9$	$ 11-20 = 9$	TIE $\rightarrow$ assign C2	C2
25	$ 25-2 = 23$	$ 25-20 = 5$	$C2 (23 > 5)$	C2

Cluster 1 = $\{2, 4, 10, 3\}$ Mean = $(2+4+10+3)/4 = 19/4 = 4.75$ Cluster 2 = $\{11, 12, 20, 25, 30\}$ Mean = $(11+12+20+25+30)/5 = 98/5 = 19.6$

ITERATION 2 — Reassign with updated centroids ($C1=4.75, C2=19.6$)

Point x	$ x - C1=4.75 $	$ x - C2=19.6 $	Assigned to	Changed from Iter 1?
2	$ 2-4.75 = 2.75$	$ 2-19.6 = 17.60$	C1	No
4	$ 4-4.75 = 0.75$	$ 4-19.6 = 15.60$	C1	No
10	$ 10-4.75 = 5.25$	$ 10-19.6 = 9.60$	C1	No
12	$ 12-4.75 = 7.25$	$ 12-19.6 = 7.60$	C1 (closer!)	YES — moved from C2
3	$ 3-4.75 = 1.75$	$ 3-19.6 = 16.60$	C1	No
20	$ 20-4.75 = 15.25$	$ 20-19.6 = 0.40$	C2	No

30	$ 30-4.75 = 25.25$	$ 30-19.6 = 10.40$	C2	No
11	$ 11-4.75 = 6.25$	$ 11-19.6 = 8.60$	C1 (closer!)	YES — moved from C2
25	$ 25-4.75 = 20.25$	$ 25-19.6 = 5.40$	C2	No

Cluster 1 = {2, 4, 10, 12, 3, 11} Mean = $(2+4+10+12+3+11)/6 = 42/6 = 7.0$ Cluster 2 = {20, 25, 30} Mean = $(20+25+30)/3 = 75/3 = 25.0$

ITERATION 3 — Check convergence (C1=7.0, C2=25.0)

Point x	$ x - C1=7 $	$ x - C2=25 $	Assigned to	Changed?
2	5	23	C1	No
4	3	21	C1	No
10	3	15	C1	No
12	5	13	C1	No
3	4	22	C1	No
20	13	5	C2	No
30	23	5	C2	No
11	4	14	C1	No
25	18	0	C2	No

New C1 = $42/6 = 7.0$ (same). New C2 = $75/3 = 25.0$ (same). **CONVERGED!**

K-MEANS FINAL RESULT (converged after 3 iterations) Cluster 1: {2, 3, 4, 10, 11, 12} Centroid = 7.0 Cluster 2: {20, 25, 30} Centroid = 25.0

Advantages: Simple, fast, scales well, guaranteed convergence. **Disadvantages:** Must specify k in advance; sensitive to initial centroids and outliers; assumes spherical clusters of similar sizes.

10 Marks

Q2. Explain the concept of DBSCAN

DBSCAN (Density-Based Spatial Clustering of Applications with Noise) (Ester et al., 1996) is a density-based clustering algorithm that groups closely packed points, marks isolated points as noise, and does NOT require specifying the number of clusters k beforehand.

Two Input Parameters

Epsilon (eps, ϵ): radius that defines the neighbourhood around each point MinPts: minimum number of points required in the ϵ -neighbourhood to qualify as a dense region (including the point itself) ϵ -neighbourhood of point p: $N_{\epsilon}(p) = \{ q \text{ in } D \mid \text{dist}(p, q) \leq \epsilon \}$

Three Categories of Points

Point Type	Definition	Condition	Example ($\epsilon=2$, MinPts=3)
Core point	Has $\geq$ MinPts points within its ϵ -neighbourhood (dense region)	$ N_{\epsilon}(p) \geq \text{MinPts}$	Point with 3+ neighbours within radius 2

Border point	Has < MinPts in ϵ -neighbourhood BUT is within ϵ of a core point	$ N_{\epsilon}(p) < \text{MinPts}$ AND p in $N_{\epsilon}(\text{some core})$	Point with 1-2 neighbours, but near a core
Noise / Outlier	Not a core point AND not within ϵ of any core point	Neither core nor border conditions met	Isolated point far from any cluster

Density Reachability Concepts

- **Directly density-reachable:** q is directly reachable from p if p is a core point AND q is in $N_{\epsilon}(p)$
- **Density-reachable:** q is density-reachable from p if there exists a chain $p=p_1, p_2, \dots, p_n=q$ where each p_{i+1} is directly reachable from p_i
- **Density-connected:** p and q are density-connected if there exists a core point o from which both p and q are density-reachable

DBSCAN Algorithm Steps

DBSCAN($D, \epsilon, \text{MinPts}$): Initialize: mark all points as UNVISITED For each unvisited point p in D : Mark p as VISITED $N = \text{FindNeighbours}(p, \epsilon)$ -- get all points within ϵ of p If $|N| < \text{MinPts}$: Mark p as NOISE -- not enough neighbours, tentatively noise Else: (p is a CORE point) Create new Cluster C
ExpandCluster($p, N, C, \epsilon, \text{MinPts}$): Add p to C For each point q in N : If q is UNVISITED: Mark q as VISITED $M = \text{FindNeighbours}(q, \epsilon)$ If $|M| \geq \text{MinPts}$: $N = N \cup M$ -- extend neighbourhood If q not in any cluster: Add q to C Return clusters and noise points

Numerical Example ($\epsilon = 3, \text{MinPts} = 3$)

Points: $A=1, B=2, C=3, D=5, E=6, F=7, G=15$ (1-dimensional)

Step 1: Check $A(=1)$: $N_3(A) = \{1,2,3,4\} \rightarrow B, C$ are within 3 $\rightarrow |N|=3 \geq \text{MinPts} \rightarrow A$ is CORE Start Cluster 1. Expand: B and C added. Step 2: Check $B(=2)$: $N_3(B) = \{1,2,3,4,5\} \rightarrow A, C, D$ are neighbours $\rightarrow |N|=4 \geq \text{MinPts} \rightarrow \text{CORE}$ D added to Cluster 1. Step 3: $D(=5)$: $N_3(D) = \{2,3,4,5,6,7,8\} \rightarrow B, C, E, F$ are neighbours $\rightarrow |N|=4 \rightarrow \text{CORE}$ E, F added to Cluster 1. Step 4: $E(=6), F(=7)$ are checked — both have enough neighbours $\rightarrow$ also CORE points Step 5: $G(=15)$: $N_3(G) = \{15\} \rightarrow |N|=1 < \text{MinPts} \rightarrow G$ is NOISE (outlier) RESULT: Cluster 1 = $\{A, B, C, D, E, F\} = \{1, 2, 3, 5, 6, 7\}$ Noise = $\{G\} = \{15\}$

Advantages and Disadvantages

Aspect	Advantages	Disadvantages
Number of clusters	Does NOT require specifying k — discovered automatically	Must choose ϵ and MinPts carefully — sensitive to these
Cluster shape	Finds arbitrarily shaped clusters (spirals, crescents, rings)	Struggles with clusters of varying density (need different ϵ for each)
Outliers	Automatically identifies and labels noise/outlier points	Border points near multiple clusters may be assigned inconsistently
Data	Works well with spatial and geographic data	High-dimensional data: distance measures lose meaning (curse of dimensionality)

10 Marks

Q3. Explain the concept of Decision Tree

Decision Tree is a supervised classification and regression model that represents decisions as a hierarchical tree structure. It is one of the most interpretable and widely used data mining methods because it produces human-readable IF-THEN rules.

Structure element	Description
Root node	Topmost node; tests the most informative attribute; has no incoming branch
Internal node	Tests a specific attribute; has exactly one incoming and two or more outgoing branches
Branch (Edge)	Represents a possible outcome (attribute value or range) of the parent node test
Leaf node	Terminal node; holds a class label (majority class of training records that reach it)

ID3 Algorithm — uses Information Gain to choose split attribute

Entropy (impurity measure): $H(S) = -\sum_i [p(i) * \log_2(p(i))]$ ($p(i)$ = fraction of class i in S) $H = 0$ means pure (all same class); $H = 1$ means maximum impurity (50-50) Information Gain of attribute A on set S : $IG(S, A) = H(S) - \sum_v [|S_v|/|S| * H(S_v)]$ ($H(S)$ = entropy before split; S_v = subset of S where A has value v) Split criterion: choose $A^* = \text{argmax}_A IG(S, A)$ -- maximum information gain

Complete Example: Play Tennis Dataset (14 training examples)

Day	Outlook	Temp	Humidity	Wind	Play Tennis?
D1	Sunny	Hot	High	Weak	NO
D2	Sunny	Hot	High	Strong	NO
D3	Overcast	Hot	High	Weak	YES
D4	Rain	Mild	High	Weak	YES
D5	Rain	Cool	Normal	Weak	YES
D6	Rain	Cool	Normal	Strong	NO
D7	Overcast	Cool	Normal	Strong	YES
D8	Sunny	Mild	High	Weak	NO
D9	Sunny	Cool	Normal	Weak	YES
D10	Rain	Mild	Normal	Weak	YES
D11	Sunny	Mild	Normal	Strong	YES
D12	Overcast	Mild	High	Strong	YES
D13	Overcast	Hot	Normal	Weak	YES
D14	Rain	Mild	High	Strong	NO

Step 1: Calculate $H(S)$ — entropy of root (9 YES, 5 NO out of 14)

$$H(S) = -(9/14) * \log_2(9/14) - (5/14) * \log_2(5/14) = -(0.643) * (-0.637) - (0.357) * (-1.485) = 0.410 + 0.530 = 0.940 \text{ bits}$$

Step 2: Calculate IG for each attribute

Attribute	Values	Weighted H(S,A)	IG = 0.940 - weighted H	Rank
Outlook	Sunny(5: 2Y,3N) H=0.971 Overcast(4: 4Y,0N) H=0.0 Rain(5: 3Y,2N) H=0.971	$(5/14)*0.971+(4/14)*0.0+(5/14)*0.971 = 0.693$	$0.940 - 0.693 = 0.246$	1st BEST
Humidity	High(7: 3Y,4N) H=0.985 Normal(7: 6Y,1N) H=0.592	$(7/14)*0.985+(7/14)*0.592 = 0.788$	$0.940 - 0.788 = 0.152$	2nd
Wind	Weak(8: 6Y,2N) H=0.811 Strong(6: 3Y,3N) H=1.0	$(8/14)*0.811+(6/14)*1.0 = 0.892$	$0.940 - 0.892 = 0.048$	3rd
Temperature	Hot(4: 2Y,2N) H=1.0 Mild(6: 4Y,2N) H=0.918 Cool(4: 3Y,1N) H=0.811	$(4/14)*1.0+(6/14)*0.918+(4/14)*0.811 = 0.911$	$0.940 - 0.911 = 0.029$	4th

Root node = Outlook (highest IG = 0.246)

Overcast branch → always YES (pure, entropy=0) → immediate LEAF node.

Sunny and Rain branches need further splitting recursively.

Step 3: Final Decision Tree structure

[Outlook] / | \ Sunny Overcast Rain | | | [Humidity] YES [Wind] / \ \ High Normal Strong Weak | | | NO
YES NO YES
Decision rules extracted: Rule 1: IF Outlook = Overcast THEN Play = YES Rule 2: IF Outlook = Sunny AND Humidity = Normal THEN Play = YES Rule 3: IF Outlook = Sunny AND Humidity = High THEN Play = NO Rule 4: IF Outlook = Rain AND Wind = Weak THEN Play = YES Rule 5: IF Outlook = Rain AND Wind = Strong THEN Play = NO

Tree Pruning: Trees can overfit — they memorise training data. **Pre-pruning:** stop splitting when improvement < threshold. **Post-pruning:** grow full tree, then replace subtrees with leaves using validation data.

Property	Decision Tree
Interpretability	Excellent — produces human-readable IF-THEN rules
Data types	Handles both categorical and continuous attributes
Preprocessing	No feature scaling or normalization needed
Speed	Fast training $O(n \log n)$ and prediction $O(\text{depth})$
Main weakness	Prone to overfitting; unstable — small data changes can produce very different trees

Quick Reference — All Formulas and Key Facts

Formula / Concept	Expression / Value
Support count	Number of transactions containing itemset X
Support (relative)	$\text{Support}(X) = \text{count}(X) / \text{total transactions}$
Confidence	$\text{Confidence}(X \rightarrow Y) = \text{Support}(X \cup Y) / \text{Support}(X)$
Lift	$\text{Lift}(X \rightarrow Y) = \text{Confidence}(X \rightarrow Y) / \text{Support}(Y)$ [> 1 = positive correlation]
Apriori property	If X is NOT frequent, no superset of X is frequent
FP-Growth vs Apriori	FP-Growth: 2 scans, no candidates. Apriori: k+1 scans, candidate generation
Entropy	$H(S) = -\sum_i [p(i) * \log_2(p(i))]$ (bits)
Information Gain	$IG(S,A) = H(S) - \sum_v [S_v / S * H(S_v)]$
Gini Index (CART)	$\text{Gini}(S) = 1 - \sum_i [p(i)^2]$
Bayes' Theorem	$P(H E) = P(E H) * P(H) / P(E)$
K-Means centroid	$\mu_k = (1/ C_k) * \sum_{xi \in C_k} xi$ (mean of cluster k)
K-Means convergence	Stop when centroids do not change between iterations
DBSCAN core point	$ N_{\epsilon}(p) \geq \text{MinPts}$
DBSCAN directly reachable	q directly reachable from p: p is core AND q in $N_{\epsilon}(p)$
Neural net weighted sum	$z = w_1 * x_1 + w_2 * x_2 + \dots + w_n * x_n + b$
ReLU activation	$f(z) = \max(0, z)$
Sigmoid activation	$f(z) = 1 / (1 + e^{(-z)})$
K-Means example result	$D=\{2,4,10,12,3,20,30,11,25\}$: $C1=\{2,3,4,10,11,12\}$ $C2=\{20,25,30\}$